Fase 4 - Activitat 7.1: Desplegant aplicacions amb contenidors Docker executant-se sobre una màquina creada amb Vagrant

0- Identificació del grup i activitat:

Curs: ASIX2
Projecte: PJ9 DevOps i CI/CD
Fase: 4
Activitat: 7.1
Grup/Individual: Individual

1.- Objectiu de l'activitat

Els objectius objectius d'aquesta activitat és desplegar una aplicació PHP utilitzant Dockers. Això vol dir:

- Crear una imatge
- Crea un contenidor a partir de la imatge
- Accedir a l'aplicació.

2.- Conceptes mínims sobre contenidors Docker

a) Avui dia hi ha 4 maneres típiques de desenvolupar i desplegar una aplicació i fer-la disponibles als usuaris finals. Podeu desplegar l'aplicació:

- Directament sobre l'equip físic.
- Sobre una màquina virtual.
- Utilitzant contenidors que s'executen sobre una màquina física
- Utilitzant contenidors que s'executen sobre una màquina virtual

Totes tenen avantatges i desavantatges però, la darrera opció ens proporciona una bona combinació de rapidesa de posada en marxa i execucio, portabilitat, eficiència en l'utilització dels recursos (CPU, RAM, espai de disc, xarxa,...), escalabilitat, aïllament (i per tant, menys risc de conflicte entre aplicacions) i seguretat.

b) Per poder treballar amb contenidors, ens cal un progamari gestor i motor de contenidors. Amb aquest programari és fàcil poder crear, posar en marxa, aturar, modificar, esborrar i utlitzar contenidors.

c) Un dels gestors i motors de contenidors més populars i més ampliament utilitzat del mercat és **Docker Engine** creat per la companyia **Docker, Inc**.

d) Alguns enllaços interessants sobre contenidors **Docker**:

- Avantatges d'un contenidor: https://www.youtube.com/watch?v=YFl2mCHdv24 (0:06 - 0:44)
- Què és un contenidor Docker?: https://www.youtube.com/watch?v=YFl2mCHdv24 (0:00 - 0:44)
- Conceptes bàsics: https://www.youtube.com/watch?v=YFl2mCHdv24 (1:57 - 2:16)
- Dockerfile i imàgens: https://www.youtube.com/watch?v=YFl2mCHdv24 (2:17 - 2:41)
- DockerHub --> https://www.youtube.com/watch?v=YFl2mCHdv24 (3:53 - 4:16)

e) Diferència entre una màquina virtual i un contenidor:

- Virtual Machine vs Docker --> https://www.youtube.com/watch?v=YFl2mCHdv24 (0:45 – 1:56)
- Virtual Machine and Container in a nutshell --> https://deshanigeethika.medium.com/docker-tutorial-a6aa5b41e3ff

3- Creació d'una màquina virtual amb Vagrant amb aprovisionament del programari per treballar amb contenidors Docker

a) Dins de la carpeta **pj9** → **f4** de la teva màquina física crea una carpeta de nom **a7**. Dins de la carpeta **a7** crea **1** directori de nom **codis**.

b) Dins de la carpeta de nom **vm** de la teva màquina física crea el següent fitxer **Vagrantfile**:

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

#########################
#### Definició de variables ####
#########################

IMATGE_BOX = "xxxx"
GESTORMV = "xxxx"

NOM_NODE = "xxxx"

NOM_BASE= "xxxx"
ID = "-xxxxx"
NOM_DOMINI = "xxxx"

MEMORIA_RAM = "xxxx"
NUM_CPUS = "xxxx"

PORT1_GUEST = "xxxx"
PORT1_HOST = "xxxx"

PORT2_GUEST = "xxxx"
PORT2_HOST = "xxxx"

PORT3_GUEST = "xxxx"
PORT3_HOST = "xxxx"

PROT = "xxxx"

CARPETA_HOST = "xxxx"
CARPETA_GUEST = "xxxx"

LLISTAT_PROGRAMARI_BASIC = "xxxx"
LLISTAT_PROGRAMARI_DEPENDENCIES_DOCKER = "xxxx"
LLISTAT_PROGRAMARI_DOCKER = "xxxx"

USUARI_VAGRANT = "vagrant"
GRUPS_USUARI_VAGRANT = "docker"

########################################
#### Creació i definició de la màquina virtual #######
########################################

Vagrant.configure("2") do |maquines|

    # Definició del l'identificador de màquina
    maquines.vm.define NOM_NODE+ID do |maquina|

        # Definició de paràmetres de la màquina virtual
        maquina.vm.provider GESTORMV do |parametres|
                parametres.name = NOM_NODE+ID
                parametres.memory = MEMORIA_RAM
                parametres.cpus = NUM_CPUS
                parametres.customize ['modifyvm', :id, '--clipboard', 'bidirectional', '--groups', '/ASIX2_PJ9']
        end
```

```
        # Definició de paràmetres del sistema de la màquina virtual
        maquina.vm.box = IMATGE_BOX
        maquina.vm.hostname = NOM_BASE+ID+NOM_DOMINI
        maquina.vm.network "forwarded_port", guest: PORT1_GUEST, host: PORT1_HOST, protocol: PROT
        maquina.vm.network "forwarded_port", guest: PORT2_GUEST, host: PORT2_HOST, protocol: PROT
        maquina.vm.network "forwarded_port", guest: PORT3_GUEST, host: PORT3_HOST, protocol: PROT
        maquina.vm.synced_folder CARPETA_HOST, CARPETA_GUEST

        ### Aprovsionament de programari i configuració###
        maquina.vm.provision "shell", inline: <<-SHELL
                sudo apt-get update -y
                sudo apt-get install -y #{LLISTAT_PROGRAMARI_BASIC}
                sudo apt-get install -y #{LLISTAT_PROGRAMARI_DEPENDENCIES_DOCKER}
                # Una ordre escrita en 2 línies. El caràcter \ serveix per indicar que la següent línia va unida a l'anterior.
                curl -fsSL https://download.docker.com/linux/debian/gpg | \
                sudo gpg --dearmor -o /usr/share/keyrings/docker.gpg
                # Final de l'ordre escrita en 2 línies
                # Una ordre escrita en 3 línies. El caràcter \ serveix per indicar que la següent línia va unida a l'anterior.
                echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker.gpg] \
                https://download.docker.com/linux/debian bookworm stable" | \
                sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
                # Final de l'ordre escrita en 3 línies
                sudo apt-get -y update
                sudo apt-get install -y #{LLISTAT_PROGRAMARI_DOCKER}
                sudo usermod -aG #{GRUPS_USUARI_VAGRANT} #{USUARI_VAGRANT}
        SHELL

    end
end
```

c) Fes les següents modificacions a les varibales del fitxer **Vagrantfile**:

- El Box utilitzat serà **debian/bookworm64**.
- El provider (software gestor de màquines virtuals) serà **virtualbox**.
- El nom de la màquina virtual dins del gestor serà a on **pj9f4a7-xxyyzz**. Recorda que **xxyyzz** són les 2 primeres lletres dels teus nom i cognoms.
- La memòria RAM assignada a la màquina virtual serà de **2048MiB**.
- La quantitat de CPUs assignades a la màquina virtual serà de 2 CPUs.
- El nom de sistema de la màquina virtual serà **pj9f4a7-xxyyzz.fjeclot.local**. Recorda que **xxyyzz** són les 2 primeres lletres dels teus nom i cognoms.
- La carpeta de la màquina física compartida amb la màquina virtual serà **codis**. Recorda que **codis** està a la mateixa carpeta que el **Vagrantfile**.
- La carpeta de la màquina virtual compartida amb la màquina física serà **/home/vagrant/codis**.
- El port **80/tcp** de la màquina virtual s'exportarà al port **8000/tcp** de la màquina física.
- El port **443/tcp** de la màquina virtual s'exportarà al port **8443/tcp** de la màquina física.
- El port **3306/tcp** de la màquina virtual s'exportarà al port **13306/tcp** de la màquina física.
- El programari bàsic serà: **net-tools**, **whois**, **aptitude**, **git**, **zip**, **unzip** i **openssh-server**
- S'han d'afegir les dependències del programari de gestió de conternidors **Docker** per funcionar i instal·lar-se tal i com ho varem veure a l'activitat **pj9f4a6.11**.
- S'ha d'afegir el programari de gestió de contenidors **Docker** tal i com ho varem veure a l'activitat **pj9f4a6.11**.
- S'ha de fer membre del grup **docker** a l'usuari **vagrant** per permetre'l utilitzi el programari de gestió de contenidors **Docker** sense que calgui que utlitzo l'ordre **sudo**.

4.- Desplegament d'una aplicació PHP per mitjà de contenidors Docker

a) Recorda que, per poder desplegar una aplicació i desplegar-la utilitzant contenidors Docker, cal seguir els següents passos:

1. Desenvolupament de l'aplicació (en PHP, Python, JavaScript, Java, C, Bash,.....).
2. Creació d'un fitxer Dockerfile.
3. Creació d'una imatge del contenidor amb el fitxer Dockerfile i l'aplicació.
4. Comprovació de que la imatge s'ha creat correctament.
5. Creació del contenidor a partir de la imatge creada.
6. Comprovació de que el contenidor s'ha creat correctament i està en execució.
7. Accés a l'aplicació per comprovar el seu funciomanent.

Ara seguirem aquests passos.

b) Surt de la màquina virtual. Dins de la màquina física:

- Accedeix a la carpeta **codis** i crea una carpeta de nom **pizzeria**.
- Dins de **pizzeria** crear una carpet de nom **app**.
- Dins de la carpeta **app**, descarrega els següents 2 fitxers:
  - **pizza.html** que es troba aquí
  - **pizza.php** que es troba aquí

Això correspon a la part de desenvolupament que normalment (però no sempre) correspon la feina que fan els desenvolupadors.

c) Dins de la carpeta **codis** de la màquina física crea un fitxer de nom **Dockerfile** amb aquest continigut:

```
# Descarrega una imatge base de contenidor Docker amb Apache2 i PHP 8.3
FROM php:8.3-apache
# El directori de treball per defecte del contenidor serà /var/www/html
WORKDIR /var/www/html
# Copia el directori app dins del directori de treball per defecte
COPY app .
# Exposa el port 80/tcp del contenidor
EXPOSE 80
```

d) Accedeix novament a la màquina virtual i comprova que dins de la carpeta **/home/vagrant/codis/pizzeria (~/codi/pizzeria)** s'ha creat:

- La carpeta **app** que té dins els fitxer **pizza.html** i **pizza.php**.
- El fitxer **Dockerfile** amb el contingut crear a l'apartat anterior

d) Crea una **imatge del contenidor Docker** de la teva aplicació executant:

```
docker build -t ipizza:1.0 .
```

i comprova que es crea la imatge **ipizza** versió **1.0** executant:

```
docker images
```

e) Crea un contenidor per de nom **cpizza1.0** que utilitzi la imatge **ipizza:1.0** i que exposi el port **80/tcp** del contenidor com a port **80/tcp** de la màquina host (en aquest cas, la màquina virtual) executant:

```
docker run --name cpizza1.0 -i -t -d -p 80:80 ipizza:1.0
```

i comprova que es s'ha creat el contenidor **cpizza1.0**, que els seu estat és **Up**, que exposa el port 80 del contenidor al port 80 de la màquina virtula i que s'ha creat a partir de la imatge ipizza:1.0. Executa:

```
docker ps -a
```

f) Comprova que la màquina virtual té el port **80/tcp** obert pel contenidor **cpizza**. Executa des de la màquina virtual:

```
sudo netstat -atupn | grep docker
```

g) Comprova que la màquina física té el port **8000/tcp** obert pel programari **Vagrant**. Executa des de la màquina física i dins del directori a on es troba el fitxer **Vagrantfile**:

```
vagrant port
```

h) Accedix a l'aplicació utilitzant l'**adreça IP de la màquina física** i recordant que el port **80/tcp** de la màquina virtual està exportat al **8000/tcp** de la màquina física. Recorda també, que per aquesta aplicació, la seva pàgina inicial és **pizza.html**. Pensa la URL d'accés a la pàgina d'inici de l'aplicació abans de cridar-la.

i) Fes una prova de funcionament. Demana 2 ingredient i comprova que el preu és **12,1€**.

5- Creació d'un nou dipòsit local de Git sincronitzat a un dipòsit de Github per pujar el fitxers de l'activitat

a) Dins del directori **a7** de la **màquina física** crea un nou dipòsit local.

b) Dins del directori **a7** , crea un fitxer **.gitignore** amb el següent contingut:

```
.gitignore
.git
.vagrant
```

e) Fes un **add** de tots els fitxers de l'activitat.

d) Fes un **commit** amb el comentari: *"Commit 1 de l'activitat a7"*.

e) Crea un dipòsit de de tipus **Públic** dins del teu compte de **Github** de nom **pj9f4a7.1**. A continuació, fes tots els passos indicats per **Github** per **sincronitzar** i **pujar** el teu dipòsit local al nou dipòsit remot.

Lliurament de l'activitat

a) Comprovacions:

- El contenidor de **Github** amb els 4 fitxers del projecte dins del seus directoris.
- La imatge de contenidor creada dins de la màquina virtual.
- El contenidor en execució.
- L'adreça IP de la màquina física.
- L'accés a l'aplicació utilitzant l'adreça IP de la màquina física i accedint al port **8000/tcp**.

b) Data límit per obtenir el 100% de la nota: **dilluns 26-10-26** fins a les **21.00**. Posteriorment la nosta serà menor.